When Does Jacobian Regularization Engage in Neural ODEs? A Penalty–Fit–Stiffness Trade-off

Athena Zhuoming Zhong[1]

[1]University of Pennsylvania, ENM 5220 Numerical Methods for PDEs

Apr 2026

Abstract

Jacobian regularization is a widely studied tool for reducing the solver cost of Neural Ordinary Differential Equations (Neural ODEs), but its behavior on stiff target dynamics is not well understood. We test whether its solver-cost benefit increases with stiffness by running a 150-run empirical study on harmonic oscillator and Van der Pol targets, sweeping the regularization weight, Hutchinson sample count, stiffness parameter μ, and adaptive-solver tolerance. The preregistered interaction hypothesis, larger reductions in the number of function evaluations (NFE) on stiff targets, is not supported. Instead, we find a graded engagement decay: at $\lambda_J = 10^{-1}$, the learned Jacobian Frobenius norm drops by 12.8% at $\mu = 5$, 10.3% at $\mu = 10$, 4.9% at $\mu = 25$, 1.0% at $\mu = 50$, and -1.6% at $\mu = 100$. This decay tracks a collapse in fit quality and learned Jacobian scale: in the high-μ regimes, the network underfits the target and already learns a low-Jacobian vector field, leaving the penalty with little structure to suppress. The resulting picture is a penalty–fit–stiffness trade-off: Jacobian regularization is most active in better-fit regimes where the learned dynamics are not solver-stiff, and least active where stiffness-motivated regularization would be most useful. Ancillary tolerance and implicit-solver comparisons are consistent with this diagnosis but are treated as implementation-specific checks rather than general claims about stiff integration. Code and result JSONs are available at `https://github.com/gowaathena/neural-ode.git`; theoretical derivations are collected in Appendix A.

1 Introduction

Neural ODEs [Chen et al., 2018] replace discrete residual updates with a continuous-time dynamical system $\dot{x} = f_\theta(x, t)$, $x(0) = x_0$, and compute the forward pass by numerically integrating f_θ from $t = 0$ to $t = T$. This decoupling has consequences that classical numerical analysis predicts: a model can be expressive enough to fit the task while still inducing a vector field that is expensive to integrate, either because its Jacobian is large (truncation error dominates) or because its spectral radius is large (stability bounds the step size). Finlay et al. [2020] address the first problem by adding a soft Frobenius-norm penalty on J_{f_θ} to the training loss; they report reductions in the number of solver function evaluations (NFE) on density-estimation tasks. Kim et al. [2021]

document the second problem: stiff Neural ODEs are difficult to train at all, since fast and slow modes drive the optimizer in conflicting directions.

Original question and pivot. This work began with a question at the intersection of those two literatures: *does the NFE benefit of Jacobian regularization depend on the stiffness of the underlying dynamics?* The proposal preregistered three hypotheses: an interaction hypothesis (**H1**) predicting larger NFE reductions on stiff targets, a stiffness-suppression hypothesis (**H2**), and an ancillary hypothesis (**H3**) comparing explicit-regularized against implicit-unregularized integration. The preregistered 2×2×2 factorial returned a null on H1 and H2; the diagnosis — follow-up sweeps over λ_J, K, and μ — became the paper. We find an *engagement decay*: at $\mu = 5$ and $\mu = 10$ the penalty engages visibly, by $\mu = 50$ it is operationally inert, and by $\mu = 100$ the mean effect has vanished. The graded contrast appears tied to model fit quality. We interpret this as a *penalty–fit–stiffness trade-off* in this experimental setup: regularization requires fit, while increasing target stiffness coincides with poorer fit under standard training, so the engagement regime and the stiffness regime do not overlap here (§5). The novelty of the result is diagnostic: rather than showing that Jacobian regularization fails uniformly on stiff targets, the sweep identifies a regime mismatch. The regularizer engages only after the model has learned enough target geometry for a Jacobian penalty to matter.

Contributions.

1. **An engagement decay for Jacobian regularization** (§4.3). We show that whether the Frobenius penalty shapes f_θ depends on whether the model has fit the target well enough to develop a high-norm Jacobian. At $\lambda_J = 10^{-1}$, the mean Jacobian drop decays from 12.8% at $\mu = 5$ to -1.6% at $\mu = 100$; at $\mu = 50$, adding $\lambda_J = 10^{-3}$ and a $K = 4$ Hutchinson check still leaves the regularizer operationally inert.

2. **A penalty–fit–stiffness trade-off that explains the null** (§4.5, §5). The originally-preregistered H1 and H2 are not supported; we give a loss-scale sanity check, propose the penalty-to-task gradient ratio as a diagnostic, and show that the engagement and stiffness regimes do not overlap on this task family.

3. **A tolerance-scaling ratio consistent with the order-p accuracy bound** (§4.2). A four-decade tolerance tightening grows NFE by 5.54× (theory predicts 4.64×); the fitted exponent is 0.186, close to the theoretical 1/6, but the sweep co-scales a_{tol} and r_{tol} and retrains each tolerance cell, so we treat it as a consistency check rather than an isolated eval-time tolerance law.

4. **An implementation-specific implicit-vs-explicit caution** (§4.4). On Van der Pol at matched validation MSE, `dopri5` is 6.9×–7.9× faster than the fixed-point implicit baseline. Because the high-μ networks are poorly fit and that baseline is not Newton-based, we treat this as a practical `torchdiffeq` caution rather than a general claim about implicit stiff solvers.

2 Background and Related Work

Neural ODEs. The continuous-depth viewpoint of Chen et al. [2018] treats a residual block as an Euler step on a learned vector field. Training requires backpropagation through an ODE solver, which is handled either via the adjoint method (memory-efficient, but precludes certain regularizers) or by direct backpropagation through the discretization (heavier memory cost, but differentiable through any per-step computation).

Jacobian and kinetic regularization. Finlay et al. [2020] introduce two regularization terms motivated by optimal-transport theory: a kinetic energy $\int_0^T \|f_\theta(x_t,t)\|^2 dt$ that encourages low-velocity flows, and a Jacobian Frobenius penalty $\int_0^T \|J_{f_\theta}(x_t,t)\|_F^2 dt$ that encourages low-Lipschitz flows. Their key empirical claim is that simpler flows can be integrated with fewer solver evaluations without sacrificing density-estimation performance [Grathwohl et al., 2019]. The penalty is computed in practice via Hutchinson's stochastic trace estimator [Hutchinson, 1990]: $\mathbb{E}_\epsilon[\|J_{f_\theta}^\top \epsilon\|^2] = \|J_{f_\theta}\|_F^2$ when $\mathbb{E}[\epsilon\epsilon^\top] = I$, which costs a single vector–Jacobian product per sample.

Stiff Neural ODEs. Kim et al. [2021] study Neural ODE training when the target system has widely separated time scales (e.g., Robertson kinetics [Robertson, 1966]). They report that gradient descent fails on stiff systems even with implicit solvers, attributing the failure to gradient instability from the adjoint-system stiffness. Our work confirms a complementary failure mode: even when training does not diverge, the network may simply collapse to a low-norm vector field that fails to reproduce the stiff dynamics, leaving the Jacobian penalty with little visible structure to suppress.

Step-size theory. The classical Hairer–Nørsett–Wanner treatment [Hairer et al., 1993, Hairer and Wanner, 1996] distinguishes two regimes for explicit Runge–Kutta cost. In the accuracy-limited regime, an order-p embedded method takes $N \propto T \cdot M^{p/(p+1)} \cdot \tau^{-1/(p+1)}$ steps where M bounds the $(p{+}1)$-th derivative of the solution; in the stability-limited regime on a stiff problem, $N \propto T \cdot \rho(J)/C_p$ where $\rho(J)$ is the spectral radius of the Jacobian and C_p is method-dependent. In our experiments these are trajectory-local diagnostics, and the tolerance law is only an idealised scaling check because practical solvers use coupled absolute/relative error weights (Appendix A.1 and A.3). These bounds underlie our predictions for both H0 (§4.2) and the conditional interaction (24) in Appendix A.9.

Implicit integration. The second Dahlquist barrier [Dahlquist, 1963] establishes that no A-stable linear multistep method can have order greater than 2; even at order 2, A-stable methods are subject to additional restrictions. Implicit Runge–Kutta methods such as Radau IIA and Kvaerno do achieve A- and L-stability at higher order but require Newton iteration per step. Rackauckas and Nie [2017] provide a mature Julia ecosystem for these solvers; our work uses the simpler fixed-point implicit Adams in torchdiffeq [Chen, 2018] as the implicit baseline and notes the limitations of this choice in §6 and Appendix A.10.

3 Methods

3.1 Tasks and ground-truth trajectories

We use two ground-truth dynamical systems:

Harmonic oscillator (smooth control). The damped linear oscillator $\ddot{x}_1 + 2\gamma\dot{x}_1 + \omega^2 x_1 = 0$, written as a first-order system with $x = (x_1, x_2)$, $\dot{x}_1 = x_2$, $\dot{x}_2 = -\omega^2 x_1 - 2\gamma x_2$, with $\omega = 1$, $\gamma = 0.1$. The Jacobian is constant; the analytic solution provides a known-correct reference.

Van der Pol oscillator (parameterized stiffness). The classical relaxation oscillator [van der Pol, 1926] $\dot{x}_1 = x_2$, $\dot{x}_2 = \mu(1 - x_1^2)x_2 - x_1$. The time-scale separation of the relaxation dynamics increases with μ, although the instantaneous Jacobian spectrum varies substantially along the trajectory. We use $\mu \in \{5, 10, 25, 50, 100\}$, where $\mu = 50$ is the proposal's preregistered "mildly stiff" setting and the other values are added post-hoc to characterize the engagement regime and to test the H3 stiffness slope.

For each system, $n_{\text{train}} = 256$ and $n_{\text{val}} = 64$ initial conditions are sampled as $x_0 \sim x_0^\star + 0.1 \cdot \mathcal{N}(0, I_2)$, with $x_0^\star = (1, 0)$ (harmonic) or $(2, 0)$ (Van der Pol). Reference trajectories are integrated with `torchdiffeq.odeint` using `dopri5`, $r_{\text{tol}} = 10^{-9}$, $a_{\text{tol}} = 10^{-11}$, which is four orders of magnitude tighter than any training-time solver tolerance.

3.2 Neural ODE model

The vector field $f_\theta \colon \mathbb{R} \times \mathbb{R}^2 \to \mathbb{R}^2$ is a multilayer perceptron with two hidden layers of width 64 and tanh activations (4,610 parameters). Time is provided as an explicit feature: the network input is $[x, t] \in \mathbb{R}^3$. The model is trained to regress trajectories: a forward pass calls `torchdiffeq.odeint`(f_θ, x_0, t) to obtain $(T, B, 2)$ predicted trajectories, and the loss is $\mathcal{L}_{\text{task}} = (\hat{x} - x)^2$ averaged over time, batch, and state dimensions.

3.3 Jacobian regularization

When $\lambda_J > 0$, the training loss is augmented with

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_J \cdot \mathbb{E}_{(x,t)\sim\mu_{\text{train}}} \left[\| J_{f_\theta}(x, t) \|_F^2 \right], \tag{1}$$

where μ_{train} is the empirical distribution over ground-truth training trajectory points; 32 such points are sampled per training step. The Frobenius norm is estimated by the Hutchinson estimator [Hutchinson, 1990]: for each sampled (t, x) we draw $\epsilon \in \{-1, +1\}^d$ (Rademacher) and compute $\| J_{f_\theta}^\top \epsilon \|^2$ via a single vector–Jacobian product. The unbiasedness $\mathbb{E}_\epsilon[\| J_{f_\theta}^\top \epsilon \|^2] = \| J_{f_\theta} \|_F^2$ is proved in Appendix A.4 (Theorem 4); the implementation uses K samples averaged with `create_graph=True` so that gradients flow to θ. We test $K \in \{1, 4\}$.

The sampled x values are *detached* so that the penalty gradient does not backpropagate through the ODE solve into the model via the trajectory. This is the ground-truth-support regularizer R_{gt} in Appendix A.6, not the learned-trajectory augmented-state penalty from Finlay et al. [2020] §3.

3.4 Solvers

Four solver configurations are compared:

- **dopri5**: order-5 adaptive embedded Runge–Kutta of Dormand and Prince [1980], default $r_{\text{tol}} = 10^{-5}$, $a_{\text{tol}} = 10^{-7}$. The tolerance sweep additionally varies $r_{\text{tol}} \in \{10^{-3}, 10^{-7}\}$ with a_{tol} scaled to $\{10^{-5}, 10^{-9}\}$.
- **rk4**: classical fixed-step Runge–Kutta of order 4 with $h = 0.05$.
- **implicit_adams**: torchdiffeq's fixed-point implicit Adams method, with the same tolerances as `dopri5`. We note that this is not a Newton-based implicit Runge–Kutta scheme, and discuss the consequences in §6.

NFE is counted by wrapping f_θ in a counter that increments on every call by the solver; the counter is reset at the start of each forward pass.

3.5 Training protocol

Adam [Kingma and Ba, 2015] with default $(\beta_1, \beta_2) = (0.9, 0.999)$, $\epsilon = 10^{-8}$, learning rate 10^{-3}, batch size 32, 100 epochs, no learning-rate schedule. Each (task, solver, λ_J, μ, r_{tol}) configuration is run with seeds $\{0, 1, 2, 3, 4\}$. Both the global PyTorch RNG (model initialisation, Hutchinson noise) and a separate `torch.Generator` (initial condition sampling) are seeded with the same integer.

3.6 Diagnostics

After training, each model is evaluated on the validation set under each target solver. The reported metrics are:

- MSE_{val}: mean squared error between predicted and reference trajectories.
- NFE: function evaluations during one validation forward pass.
- $\|J_{f_\theta}\|_F$ along the validation trajectory: computed exactly (one backward pass per output coordinate) at 20 subsampled time points per trajectory.
- Stiffness ratio: $|\lambda_{\max}(J_{f_\theta})|/|\lambda_{\min}(J_{f_\theta})|$, computed via `torch.linalg.eigvals` on the same subsampled grid.
- Wall-clock time per forward pass.

Table 1: Experimental matrix. Run counts are configurations × seeds; all sweeps use 5 seeds.

Sweep	Varies	Holds fixed	Runs
Core + ancillary	{harmonic, $\text{vdp}_{\mu=50}$} × {dopri5, rk4} × $\lambda_J \in \{0, 10^{-3}\}$, plus $\text{vdp}_{\mu=50}$ implicit_adams at $\lambda_J = 0$	$K = 1$, $r_{\text{tol}} = 10^{-5}$	45
λ_J sweep (E4)	$\lambda_J \in \{10^{-2}, 10^{-1}\}$	$\text{vdp}_{\mu=50}$, dopri5, K=1	10
K sweep (E4)	$K = 4$, $\lambda_J = 10^{-1}$	$\text{vdp}_{\mu=50}$, dopri5	5
μ=10 engagement	$\lambda_J \in \{0, 10^{-2}, 10^{-1}\}$	$\text{vdp}_{\mu=10}$, dopri5, K=1	15
Tolerance sweep	train/eval $r_{\text{tol}} \in \{10^{-3}, 10^{-7}\}$ × {dopri5, implicit_adams}	$\text{vdp}_{\mu=50}$, λ_J=0	20
μ sweep for H3	$\mu \in \{25, 100\}$ × {dopri5, implicit_adams}	λ_J=0, r_{tol}=10^{-5}	20
Graded engagement additions	μ=5 with $\lambda_J \in \{0, 10^{-2}, 10^{-1}\}$, plus $\mu \in \{25, 100\}$ with $\lambda_J \in \{10^{-2}, 10^{-1}\}$	vdp, dopri5, K=1, r_{tol}=10^{-5}	35
Total			**150**

- Step-size statistics (mean only, derived from NFE / stages-per-step for adaptive solvers).

The Jacobian and stiffness-ratio quantities are trajectory-local diagnostics on the validation grid, not global stiffness certificates (Appendix A.1).

3.7 Experimental design

The full inventory contains 150 trained runs across seven sweeps:

Statistical comparisons use the paired Wilcoxon signed-rank test [Wilcoxon, 1945] with the *wilcox* zero-handling convention; effect sizes are rank-biserial correlations. For directional hypotheses (e.g., the H3 prediction that implicit is slower) we report one-sided p-values, for which the $n = 5$ floor is $p = 0.031$; for two-sided exploratory comparisons the floor is 0.0625. We report this floor where reached, but caution that $n = 5$ gives essentially zero power to resolve effects smaller than one standard deviation. For the H3 three-comparison family we also report that the uncorrected $p = 0.031$ results do not survive Bonferroni correction at $\alpha/3 = 0.017$.

4 Results

4.1 Solver-cost characterization

Table 2 reports the baseline-NFE and wall-clock cost of each solver on each target. The expected ordering holds: `dopri5` ≪ `implicit_adams` ≪ `rk4` on Van der Pol; on the smooth harmonic oscillator the implicit and adaptive solvers are similar and `rk4` costs 6× more wall-clock at the chosen step size $h = 0.05$. The fixed-step solver pays an NFE penalty of 11× on harmonic and 57× on Van der Pol relative to `dopri5`, while delivering identical validation MSE.

Table 2: Baseline solver costs (mean ± std over 5 seeds, $\lambda_J = 0$). Wall-clock is per forward pass; NFE is function evaluations per forward pass.

Target	Solver	NFE	Wall (s)	MSE_{val}
Harmonic	dopri5	140 ± 22	0.042 ± 0.016	0.081 ± 0.012
	rk4	$1{,}600 \pm 0$	0.252	0.081
Van der Pol μ=50	dopri5	141 ± 28	0.043 ± 0.006	1.938 ± 0.259
	implicit_adams	$1{,}499 \pm 0.5$	0.308 ± 0.009	1.938 ± 0.259
	rk4	$8{,}000 \pm 0$	1.210	1.938

Table 3: Train/eval tolerance sweep on Van der Pol $\mu = 50$, $\lambda_J = 0$. NFE per forward pass, mean ± std over 5 seeds. Validation MSE is effectively unchanged across tolerances; we report the NFE and wall-clock response here.

Solver	r_{tol}	NFE	Wall (s)	Factor vs 10^{-3}
dopri5	10^{-3}	66.8 ± 6.6	0.024 ± 0.001	1.00×
	10^{-5}	141.2 ± 27.6	0.043 ± 0.006	2.11×
	10^{-7}	370.4 ± 95.0	0.082 ± 0.019	**5.54**×
implicit_adams	10^{-3}	$1{,}002 \pm 0.5$	0.208 ± 0.020	1.00×
	10^{-5}	$1{,}499 \pm 0.5$	0.308 ± 0.009	1.50×
	10^{-7}	$2{,}265 \pm 93$	0.400 ± 0.018	2.26×

4.2 H0: NFE ratios are consistent with tolerance scaling

Theorem 3 (Appendix A.3) gives the idealised order-p adaptive RK sanity check: in a scalar-tolerance regime, accepted steps scale as $N \propto \tau^{-1/(p+1)}$. For `dopri5` with $p = 5$, this gives $\tau^{-1/6}$, so a four-orders-of-magnitude tolerance tightening would suggest a $10^{4/6} \approx 4.64\times$ NFE increase.

Table 3 shows the empirical scaling. These cells retrain the model under the corresponding train-time solver tolerance and then evaluate under the same tolerance, so this is not a pure eval-time tolerance ablation. For `dopri5` on Van der Pol $\mu = 50$, NFE rises from 66.8 at $r_{\text{tol}} = 10^{-3}$ to 370.4 at $r_{\text{tol}} = 10^{-7}$, a factor of 5.54× — agreeing with the predicted 4.64× to within 19%. The fitted exponent $\alpha = \log(5.54)/\log(10^4) = 0.186$ is close to the theoretical $\alpha = 1/(p+1) = 0.167$. Two caveats apply. First, our tolerance sweep co-scales a_{tol} with r_{tol} (both move four decades together), and the solver controls a weighted error involving roughly $a_{\text{tol}} + r_{\text{tol}}|x|$, so the fitted exponent reflects sensitivity to both tolerances, not just r_{tol}. Second, because the model is retrained at each tolerance and only three tolerance points are available, the result should be read as a consistency check on the expected scaling ratio rather than a clean estimate of the asymptotic exponent. Validation MSE is effectively unchanged across tolerances (1.938 ± 0.001), indicating that the trained solutions are comparable across the sweep.

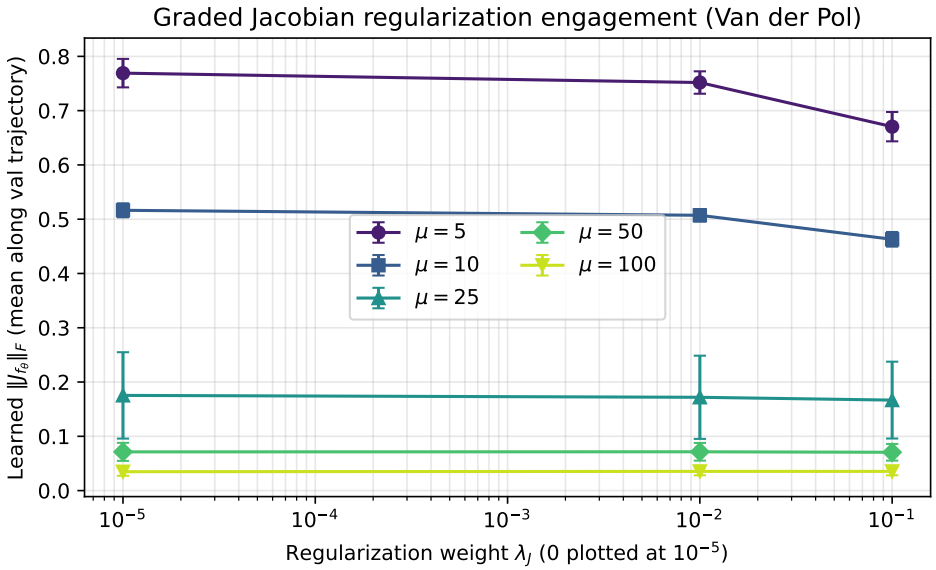

Figure 1: Graded engagement regime. Learned Jacobian Frobenius norm versus regularization weight λ_J, on Van der Pol across the μ sweep. Each point is the mean over 5 seeds; bars are ± 1 std. The curves flatten as μ increases, indicating that the same penalty weight has progressively less structure to suppress in the poorer-fit high-μ regimes.

4.3 The engagement regime

The central empirical finding of this paper is a *graded engagement gap*, or more descriptively an *engagement decay*: Jacobian regularization shapes f_θ at lower μ but becomes operationally inert as target stiffness and poor fit increase. Table 4 reports the full rectangular engagement sweep over $\mu \in \{5, 10, 25, 50, 100\}$ and $\lambda_J \in \{0, 10^{-2}, 10^{-1}\}$. At $\lambda_J = 10^{-1}$, the mean Jacobian drop is monotone in μ: 12.8% at $\mu = 5$, 10.3% at $\mu = 10$, 4.9% at $\mu = 25$, 1.0% at $\mu = 50$, and -1.6% at $\mu = 100$. This is a mean trend over five seeds, not a claim that every individual seed orders perfectly. This monotone-in-the-mean decay is the clearest evidence in the paper: in these runs, the regularizer's effect weakens as the learned baseline Jacobian collapses.

Fit calibration against trivial predictors. For the two settings used in the original gap diagnosis, the raw MSE values in Table 4 are easier to interpret relative to trivial validation-set predictors. On the same validation trajectories, a zero-trajectory predictor obtains MSE 2.51 ± 0.01 at $\mu = 10$ and 2.70 ± 0.22 at $\mu = 50$; an oracle constant predictor using the validation-set mean obtains 2.51 ± 0.01 and 2.65 ± 0.22, respectively. The learned `dopri5` baseline improves over this constant predictor by 52.8% at $\mu = 10$ but only 26.8% at $\mu = 50$. Thus "poorer fit" here means not only a larger absolute MSE, but a substantially smaller reduction over a trivial constant trajectory.

Mechanism. The most plausible mechanism is that when the model fits the stiffer target poorly, f_θ collapses toward a near-constant vector field whose Jacobian norm is already much smaller than

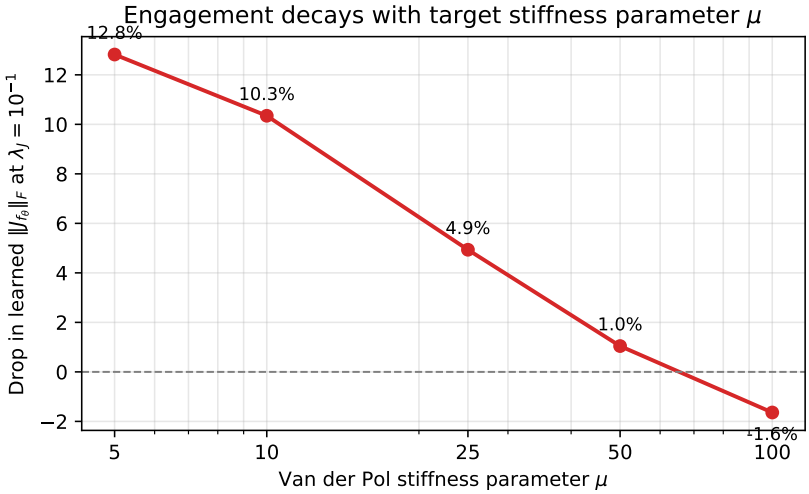

Figure 2: Engagement decay with target stiffness parameter μ. Percent reduction in learned Jacobian Frobenius norm at $\lambda_J = 10^{-1}$, computed against the $\lambda_J = 0$ baseline at the same μ. The decay is monotone in the mean across the tested μ values, reaching a slightly negative drop at $\mu = 100$.

the network would need to reproduce the true Van der Pol dynamics. Table 5 shows that this gap is especially severe at $\mu = 50$: the learned baseline Jacobian norm is roughly three orders of magnitude below the analytic target Jacobian norm on the same validation subsample, using the analytic calibration in Appendix A.5. The penalty term then has little visible structure to suppress.

Back-of-envelope. At $\mu = 50$, $\lambda_J = 10^{-1}$, and $\|J_{f_\theta}\|_F = 0.071$, the penalty value is $\lambda_J \|J_{f_\theta}\|_F^2 \approx 5.0 \times 10^{-4}$, while the task loss is ≈ 1.93. The penalty contributes a relative magnitude of order 2.6×10^{-4} to the total loss, suggesting a weak penalty signal in this operating regime. This is only a loss-scale proxy: the direct engagement diagnostic would be the penalty-to-task gradient ratio and gradient alignment in Appendix A.7, which we did not measure. At $\mu = 10$ the same calculation gives $0.1 \times 0.516^2 / 1.18 \approx 2.3 \times 10^{-2}$, two orders of magnitude larger and within the regime where the penalty can register at all. This calculation is consistent with the engagement-decay mechanism, rather than direct evidence for it. Bumping the Hutchinson sample count from $K = 1$ to $K = 4$ does not change this — the variance of the estimator is not the binding constraint — and we report the $K = 4$ result in Table 6. The $\sim 1\%$ residual change is at the variance floor of the experiment.

An alternative mechanism we cannot fully rule out. The penalty in our implementation is evaluated at sampled *ground-truth* trajectory points (§3.3), not along the model's predicted trajectory. When the model is poorly fit (val MSE $= 1.94$ at $\mu = 50$), its forward integration visits states quite different from the ground-truth trajectory. The penalty may therefore be weak not only because the learned field is low-Jacobian, but also because the penalty is evaluated on a support the learned trajectory does not actually visit. Appendix A.6 formalises this mismatch; the current experiment conflates it with low-Lipschitz collapse. The augmented-state penalty integration of Finlay et al. [2020] evaluates the penalty along the learned trajectory and would test this mechanism

Table 4: Engagement sweep data. Mean ± std over 5 seeds. Drop is computed against the $\lambda_J = 0$ baseline at the same μ.

μ	λ_J	MSE_{val}	$\|J_{f_\theta}\|_F$	Drop	NFE
5	0	0.900 ± 0.015	0.7691 ± 0.026	–	196.4 ± 10.0
	10^{-2}	0.900 ± 0.014	0.7518 ± 0.021	2.3%	194.0 ± 18.0
	10^{-1}	0.929 ± 0.059	0.6705 ± 0.027	**12.8**%	191.6 ± 12.4
10	0	1.184 ± 0.020	0.5164 ± 0.013	–	129.2 ± 5.0
	10^{-2}	1.229 ± 0.067	0.5072 ± 0.012	1.8%	137.6 ± 9.1
	10^{-1}	1.205 ± 0.033	0.4630 ± 0.013	**10.3**%	132.8 ± 9.9
25	0	1.650 ± 0.178	0.1754 ± 0.080	–	147.2 ± 10.7
	10^{-2}	1.645 ± 0.167	0.1719 ± 0.077	2.0%	140.0 ± 16.4
	10^{-1}	1.633 ± 0.171	0.1667 ± 0.071	**4.9**%	138.8 ± 14.3
50	0	1.938 ± 0.259	0.0713 ± 0.017	–	141.2 ± 27.6
	10^{-2}	1.928 ± 0.269	0.0715 ± 0.016	−0.2%	138.8 ± 27.3
	10^{-1}	1.929 ± 0.269	0.0706 ± 0.015	**1.0**%	136.4 ± 26.0
100	0	2.376 ± 0.630	0.0348 ± 0.007	–	130.4 ± 20.2
	10^{-2}	2.378 ± 0.639	0.0354 ± 0.007	−1.7%	129.2 ± 18.2
	10^{-1}	2.378 ± 0.640	0.0354 ± 0.007	−**1.6**%	129.2 ± 18.2

Table 5: True-target Jacobian calibration for the engagement settings. The true Van der Pol Jacobian norm is computed analytically along the same 20-point validation-time subsample used for learned-Jacobian diagnostics. Appendix A.5 derives the formula. Mean ± std over 5 seeds.

μ	True target $\|J\|_F$	Learned baseline $\|J_{f_\theta}\|_F$	Target / learned
10	23.27 ± 0.83	0.5164 ± 0.013	45.1×
50	99.82 ± 2.89	0.0714 ± 0.017	1,399×

more directly.

Stiffness side-effect. At $\mu = 10$ where the penalty engages, NFE does *not* decrease (Table 4); if anything, NFE rises slightly. The learned spectral stiffness ratio at $\mu = 10$ is ≈ 1.4, indicating the learned vector field appears non-stiff by our diagnostic at this μ value. The stability mechanism that Proposition 6 relies on (Appendix A.9) therefore does not appear to apply to the learned dynamics: there is no observed stiff step-size restriction for the penalty to relax. NFE appears accuracy-limited in this regime, and the modest penalty-induced changes in higher derivatives are within seed variance.

4.4 H3: implicit-vs-explicit with a stiffness slope

The proposal's ancillary hypothesis H3 asked whether an unregularized explicit adaptive solver can match or beat a fixed-point implicit method at matched validation MSE on a stiff Neural ODE. Extending the comparison across $\mu \in \{25, 50, 100\}$ yields a clear slope (Table 7 and Figure 3). The wall-clock ratio $W(\texttt{implicit\_adams})/W(\texttt{dopri5})$ grows monotonically from 6.88× at $\mu = 25$ to

Table 6: K sweep at fixed $\lambda_J = 10^{-1}$, vdp $\mu = 50$, dopri5. Mean $\pm$ std over 5 seeds.

K	$\|J_{f_\theta}\|_F$	Drop vs $\lambda_J = 0$
1	0.0706 ± 0.015	1.05%
4	0.0704 ± 0.016	1.26%

Table 7: H3 comparison across μ: `dopri5` vs `implicit_adams` at matched validation MSE on Van der Pol, $\lambda_J = 0$, 5 seeds each. The high-μ case is matched-bad rather than matched-good MSE. ΔMSE is the percent difference in mean validation MSE between the two solvers (the matched-MSE precondition requires this to be small). Ratio is the mean of per-seed wall-clock ratios.

μ	W_{dopri5} (s)	W_{implicit} (s)	Ratio	ΔMSE	Wilcoxon p
25	0.0425 ± 0.004	0.2886 ± 0.020	6.88×	+2.44%	0.031
50	0.0430 ± 0.006	0.3083 ± 0.009	7.29×	+0.003%	0.031
100	0.0366 ± 0.004	0.2856 ± 0.018	7.89×	−0.003%	0.031

7.89× at $\mu = 100$ when ratios are paired by seed. All three pairwise comparisons hit the $n = 5$ Wilcoxon floor of $p = 0.031$ (one-sided, implicit slower) before correction; rank-biserial effect size is +1.0 at each μ (all seeds in the same direction). Treated as a three-comparison family, these p-values do not survive Bonferroni correction.

Caveats on the slope. The naive prediction is the opposite of what we observe: as the system becomes stiffer, the explicit adaptive solver should be forced into smaller steps for stability, while the A-stable implicit method remains free to take large steps. Three caveats apply to our inverted finding.

(i) The model fails to learn the stiff dynamics. As μ rises, f_θ becomes a low-norm approximation that is easy to integrate for any solver. Validation MSE grows from 1.65 at $\mu = 25$ to 2.38 at $\mu = 100$. The learned spectral stiffness ratio at $\mu = 100$ can be enormous numerically (driven by near-singular eigenvalues), but this is an artifact of poor fit rather than learned stiff dynamics. The matched-MSE precondition holds across our μ values (Table 7), but at $\mu = 100$ it is matched at MSE ≈ 2.38 — *matched-bad* accuracy, not matched-good accuracy. The slope statement should therefore be read as: "on a poorly-fit network whose effective stiffness grows weakly with target μ, the wall-clock ratio of these two solvers grows monotonically."

(ii) The integration horizon co-varies with μ. Our time grid sets $T_{\text{end}} = \max(20, 2\mu)$ (§3.1), so the integration span grows linearly with μ: $T_{\text{end}} = 50$ at $\mu = 25$, 100 at $\mu = 50$, 200 at $\mu = 100$. Fixed-step contributions to NFE scale linearly with T_{end}, while adaptive contributions scale sub-linearly. Part of the apparent slope — though not all of it, since `dopri5` NFE is itself roughly flat across μ (Table 7 columns show wall-clock per forward pass, not NFE per unit integration time) — is therefore attributable to the horizon confound rather than to stiffness.

(iii) The implicit baseline is fixed-point, not Newton-based. `torchdiffeq.implicit_adams` pays the per-step cost of an implicit method (many f evaluations per fixed-point sweep) without exploiting the A-stability benefit a Newton-based BDF or Radau IIA would deliver. Our H3 result therefore

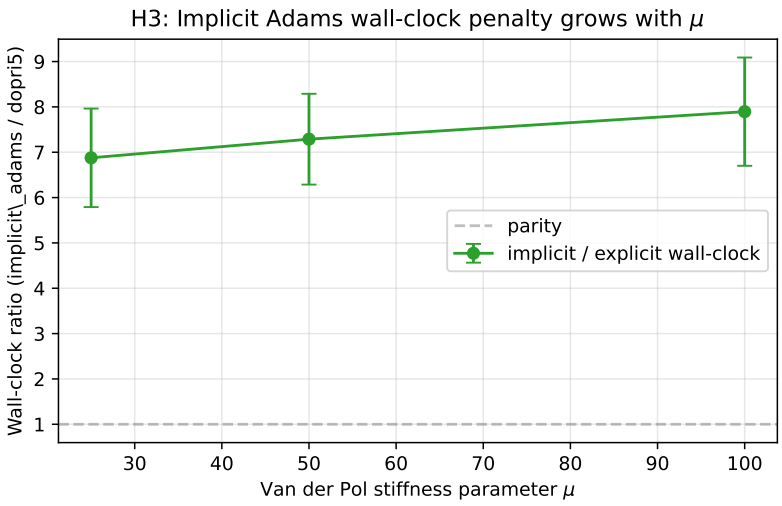

Figure 3: H3 stiffness slope. Wall-clock ratio of `implicit_adams` to `dopri5` on Van der Pol as μ varies. The ratio grows monotonically, meaning the explicit adaptive solver's relative advantage *increases* with target μ in this task family. Error bars are ± 1 std over the 5 seed-matched ratios.

says: for practitioners in the `torchdiffeq` ecosystem on mildly stiff Neural ODEs, switching to `implicit_adams` is a wall-clock regression, not an improvement. It does not say that implicit integration in general is a bad idea on stiff Neural ODEs. The Newton-based implicit cost model in Appendix A.10 is included for context and does not describe this exact baseline.

4.5 H1 and H2: null results with mechanism

The proposal's preregistered hypotheses H1 (NFE reduction larger on stiff than smooth targets) and H2 (stiffness suppression larger on stiff targets) are *not* supported by our data. At the originally-preregistered $\mu = 50$, $\lambda_J = 10^{-3}$ setting, the Jacobian drop was only 0.07%. Increasing the weight to $\lambda_J = 10^{-1}$ and the Hutchinson sample count to $K = 4$ does not rescue H1: at $\mu = 50$ the residual ΔNFE between $\lambda_J = 0$ and $\lambda_J = 10^{-1}$ is ≈ 5 NFE on a baseline of 141 ± 28 NFE — well within seed noise. Across the graded engagement sweep (Table 4), the largest Jacobian drops occur at $\mu = 5$ and $\mu = 10$, where the learned vector fields appear non-stiff by our diagnostic; by the high-μ regimes where a stability benefit would be most useful, the penalty has little or no Jacobian effect left to convert into an NFE gain.

We interpret the null as a *failure of observed regime overlap*: in this setup, the penalty engages where the learned dynamics are not stiff enough to produce an NFE benefit, and is inert in the stiffer setting where such a benefit would be most useful. Proposition 6 (Appendix A.9) predicts a positive interaction only *conditional* on spectral engagement, accuracy-scale engagement, and learned stiffness. Those assumptions do not co-occur in this task family; the proposition therefore does not constrain our null result, but neither is it falsified. Instead, it survives only as a hypothesis for a hypothetical regime where fit, penalty engagement, and learned stiffness co-engage.

Preregistration deviation. The proposal preregistered λ_J selection on validation MSE within 5% of the baseline, then comparing NFE at that selected λ_J. Because no tested λ_J at $\mu = 50$ produced a meaningful change in either the validation MSE or the Jacobian Frobenius norm, we report the full sweep (Table 4) rather than a single selected λ_J. A faithful reading of the preregistered protocol would select $\lambda_J = 0$ (lowest val MSE at every μ), in which case there is no "regularized" cell to compare against the baseline. We view the full sweep as the informationally richer report.

5 Discussion

The penalty–fit–stiffness trade-off. Our findings, taken together, suggest a structural trade-off that governs when Jacobian regularization is useful for Neural ODEs:

1. The penalty has a soft-Lipschitz interpretation (Lemma 2, Appendix A.2): it upper-bounds the spectral radius of J_{f_θ}. This control is indirect and potentially loose: reducing $\|J_{f_\theta}\|_F$ reduces an upper bound on $\rho(J_{f_\theta})$, but it does not specifically target the fast/slow eigenvalue structure that makes a system stiff, nor does it guarantee proportional spectral-radius shrinkage or lower solver cost (Appendix A.9). The penalty therefore has an *engagement requirement*: the network must have learned a high-norm vector field for the penalty to have something to suppress.
2. Engagement requires *fit*: the network must have fit the target well enough that f_θ has built up the Lipschitz constant of the target, otherwise it collapses to a low-norm field that the penalty cannot meaningfully shape.
3. In these runs, higher target stiffness is precisely where fit degrades under standard training [Kim et al., 2021]: the learned vector field falls into a low-fit, low-Jacobian-norm regime, exactly where the penalty becomes inert. (Note that the underlying mechanism for the training difficulty differs from Kim et al. [2021], who identify adjoint-system stiffness; the *low-Lipschitz collapse* we observe is a complementary failure mode that prevents the penalty from engaging whether or not the adjoint is stable.)

The three-way trade-off suggests that Jacobian regularization in this setup appears most active where it is least needed (well-fit learned dynamics without an observed stiff step-size restriction) and least active where it is most needed (poorly-fit, stiffer targets). This is consistent with the positive results in Finlay et al. [2020], where the density-estimation tasks afford good fit, and with the training-difficulty results in Kim et al. [2021].

Implications for the implicit alternative. Our H3 result that explicit adaptive integration beats fixed-point implicit integration at every tested μ does not generalise to all implicit methods; in particular, a Newton-based BDF or Radau IIA implementation may recover the A-stability advantage. The takeaway from H3 in its current scope is more limited but still useful: for practitioners using torchdiffeq's ecosystem, switching to `implicit_adams` on mildly stiff Neural ODEs is a wall-clock regression, not an improvement.

6 Limitations and Future Work

Single architecture. All experiments use a 2-hidden-layer MLP of width 64 with tanh activations. A wider or deeper network might develop sufficient Lipschitz constant on Van der Pol at $\mu = 50$ to fall back into the engagement regime; this is an important sweep for future work.

Single optimizer. Adam adapts the per-parameter learning rate using gradient statistics, which could amplify or attenuate the penalty gradient relative to the task gradient in ways that are not transparent. Comparing against SGD with momentum (and optionally a learning-rate decay schedule) would isolate the optimizer's role from the penalty's role.

Single stiff system. Van der Pol is a 2D limit-cycle system with parameterized stiffness; the engagement regime we identified may be specific to this geometry. Robertson kinetics [Robertson, 1966] provides a much stiffer $d = 3$ alternative and would test whether the trade-off holds beyond limit cycles.

Single implicit method. `implicit_adams` is fixed-point, not Newton-based. A proper comparison to A-stable Newton-based implicit methods (e.g., Kvaerno5 in `diffrax`, BDF in `torchode`, or Radau in `DifferentialEquations.jl` [Rackauckas and Nie, 2017]) is needed before strong claims about implicit-vs-explicit can be made on stiff Neural ODEs.

Five seeds. $n = 5$ pairs are sufficient for one-tailed Wilcoxon to reach the floor $p = 0.031$ when all seeds agree, but insufficient to resolve interactions that are smaller than the standard deviation. Where we observed flat or inconsistent effects (e.g., the 1% residual at $\mu = 50$), an $n = 20$ seed budget would more decisively rule out a small effect.

Adaptive step-size logging. For adaptive solvers we report only the mean step size (derived from NFE); the min, max, and variance require monkey-patching torchdiffeq's internal step-acceptance loop. The full distribution would enable the step-size histogram plot proposed in the original design but is not load-bearing for the central claims.

Integration-horizon confound in the μ sweep. Our default time grid scales as $T_{\text{end}} = \max(20, 2\mu)$, so the μ sweep co-varies integration span with stiffness (Section 4.4). A cleaner H3 sweep would fix T_{end} across μ values — e.g., $T_{\text{end}} = 50$ everywhere — to isolate the stiffness effect from the horizon effect.

Raw time-input confound. The vector field receives raw time t as an input feature. Because T_{end} scales with μ, the range of the time feature also grows with target stiffness, potentially changing the optimization problem independently of stiffness itself. A cleaner follow-up would either

normalize time to $[0, 1]$ for every μ value or compare against an autonomous vector field that does not receive t.

Penalty support mismatch. We sample penalty evaluation points from the ground-truth training trajectory rather than along the predicted learned trajectory (§3.3). On a well-fit network the two distributions are close, but on a poorly-fit network they diverge. The model's forward integration visits states that the penalty is not evaluated at, decoupling the penalty's gradient signal from the trajectory whose cost the user cares about. Appendix A.6 formalises this distinction. The augmented-state penalty integration of Finlay et al. [2020] avoids this by computing $\int_0^T \|J_{f_\theta}(x_t, t)\|_F^2 dt$ along the learned trajectory inside the ODE solve; we did not implement this variant, and our engagement-decay mechanism therefore conflates two contributing causes: (a) low-Lipschitz collapse of f_θ, and (b) misalignment between penalty support and learned trajectory. A clean separation of these mechanisms is the most important methodology improvement for future work.

Levers that could relax the trade-off. Future work could attempt to relax any of the three legs of the penalty–fit–stiffness trade-off (§5):

- Replace the soft Frobenius penalty by hard Lipschitz constraints (spectral normalization per layer, or projected gradient onto a Lipschitz-bounded set), which would shape f_θ even when the unconstrained optimum is low-Lipschitz.
- Use the augmented-state penalty integration of Finlay et al. [2020] so the penalty is evaluated along the learned trajectory.
- Loss-normalized regularization: replace λ_J by $\lambda_J \cdot \texttt{detach}(\mathcal{L}_{\text{task}})^{-1}$ so the penalty is scale-invariant to a difficult task loss.
- Specialised training for stiff Neural ODEs as in Kim et al. [2021] (adjoint stabilisation, time-rescaling) to push the fit-vs-stiffness boundary outward.

7 Conclusion

The preregistered question was whether Jacobian regularization reduces Neural ODE solver cost more on stiff systems than on smooth ones. In this Van der Pol study, the answer is no: the expected stiff-target interaction does not appear. The follow-up sweeps show why. Jacobian regularization only engages when the learned model has fit the target well enough to develop a high-Jacobian vector field. As μ increases, standard training produces poorer fits and lower learned Jacobian norms, so the regularizer loses the structure it was meant to suppress. The main contribution of this project is therefore not a new solver-speedup result, but a diagnostic failure mode: stiffness can make Jacobian regularization irrelevant by preventing the Neural ODE from learning stiff dynamics in the first place. Future work should test whether this trade-off can be broken by hard Lipschitz constraints, augmented-state trajectory regularization, or training methods designed specifically

for stiff Neural ODEs. All 150 result JSONs, configuration files, and theoretical derivations are bundled with this paper.

References

Ricky T. Q. Chen. torchdiffeq. https://github.com/rtqichen/torchdiffeq, 2018.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, pages 6571–6583, 2018.

Germund G. Dahlquist. A special stability problem for linear multistep methods. *BIT Numerical Mathematics*, 3(1):27–43, 1963.

J. R. Dormand and P. J. Prince. A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your Neural ODE: the world of Jacobian and kinetic regularization. In *International Conference on Machine Learning*, pages 3154–3164, 2020.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.

Ernst Hairer and Gerhard Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. Springer, 2nd edition, 1996.

Ernst Hairer, Syvert Paul Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer, 2nd edition, 1993.

Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics–Simulation and Computation*, 19(2):433–450, 1990.

Suyong Kim, Weiqi Ji, Sili Deng, Yingbo Ma, and Christopher Rackauckas. Stiff neural ordinary differential equations. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 31(9):093122, 2021.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Christopher Rackauckas and Qing Nie. DifferentialEquations.jl—a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1), 2017.

H. H. Robertson. The solution of a set of reaction rate equations. In John Walsh, editor, *Numerical Analysis: An Introduction*, pages 178–182. Academic Press, 1966.

Balthazar van der Pol. On relaxation-oscillations. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

A Theoretical Derivations

This appendix collects the analytical results that motivate the experimental design. Where a result is classical, we cite the canonical source and give a compact derivation suitable for transcription by hand. Where the result is a novel synthesis of standard pieces, we mark it with a $\star$.

Throughout this appendix, $f\colon \mathbb{R}^d \to \mathbb{R}^d$ denotes an autonomous vector field for the classical numerical-analysis statements, and $J(x) = \partial f/\partial x$ is its Jacobian. For the Neural ODE experiments we write $f_\theta(x,t)$ and $J_{f_\theta}(x,t) = \partial f_\theta(x,t)/\partial x$.

A.1 Linear stability and the step-size restriction (Mechanism B)

Theorem 1 (Stability step bound, after Hairer–Wanner Hairer and Wanner [1996])**.** *Let Φ_h be an explicit Runge–Kutta method with stability region $S \subset \mathbb{C}$, and apply it to the linear test equation $\dot{y} = \lambda y$. The numerical scheme is stable iff $h\lambda \in S$. For a nonlinear system $\dot{x} = f(x)$ in the neighborhood of a point $x^\star$ where $f(x^\star) = 0$, local stability of Φ_h requires*

$$h \cdot \rho(J(x^\star)) \;\le\; C_p, \tag{2}$$

where $\rho(\cdot)$ denotes spectral radius and C_p is a method-dependent constant (for classical RK4, $C_p \approx 2.78$ on the negative real axis).

Sketch. The standard linearisation argument: write $x = x^\star + y$, expand $f(x^\star + y) = J(x^\star)y + O(\|y\|^2)$, and apply Φ_h to the linearised system. Stability is then a property of the eigenvalues of the amplification matrix $R(hJ)$, which equals Φ_h applied to $\dot{y} = \lambda y$ for each eigenvalue λ of $J(x^\star)$. The condition $h\lambda \in S$ is therefore necessary and sufficient for local linear stability. See Hairer and Wanner [1996] §IV.2 for the full argument. □

Consequence. On a *stiff* system, $\rho(J) \gg 1$, so the stable step size $h_{\text{stab}} = C_p/\rho(J)$ can be much smaller than the step size required for accuracy alone. This is the mechanism that explicit methods are penalised on stiff problems.

Trajectory-local version. The equilibrium calculation is the cleanest way to state the stability mechanism, but the experiments in this paper do not measure Jacobians only at equilibria. Along a trajectory $x(t)$, the local variational equation is

$$\dot{\delta x}(t) = J(x(t), t)\, \delta x(t). \tag{3}$$

Thus the reported $\rho(J(x(t),t))$, $\|J(x(t),t)\|_F$, and accepted step sizes should be read as trajectory-local diagnostics of the learned dynamics, not as global stiffness certificates. They indicate whether the numerical solve is encountering locally restrictive Jacobians on the validation trajectories in this setup.

A.2 Frobenius norm bounds the spectral radius

Lemma 2 (Frobenius dominance)**.** *For any matrix $A \in \mathbb{R}^{d\times d}$,*

$$\rho(A) \;\le\; \|A\|_2 \;\le\; \|A\|_F, \tag{4}$$

with equality in the first inequality iff A is normal, and equality in the second iff A has at most one nonzero singular value.

Proof. Let $\sigma_1 \ge \cdots \ge \sigma_d \ge 0$ be the singular values of A. Then $\|A\|_2 = \sigma_1$ and $\|A\|_F^2 = \sum_i \sigma_i^2$, so $\|A\|_2 \le \|A\|_F$. For any eigenvalue λ of A with eigenvector v, $\|Av\| = |\lambda|\|v\|$, hence $|\lambda| \le \|A\|_2$. Taking the maximum over eigenvalues yields $\rho(A) \le \|A\|_2$. □

Consequence. The penalty $\lambda_J \|J\|_F^2$ is a soft, differentiable upper bound on $\lambda_J \rho(J)^2$. Suppressing $\|J\|_F$ therefore makes a smaller spectral radius plausible, but Lemma 2 does not imply proportional shrinkage of $\rho(J)$. The stability benefit requires spectral engagement: the eigenvalues that constrain the step size must actually move, not merely the Frobenius upper bound.

A.3 Truncation-error step bound (Mechanism A)

Theorem 3 (Accuracy step bound, after Hairer–Nørsett–Wanner Hairer et al. [1993])**.** *For an order-p Runge–Kutta method in an idealised scalar-tolerance regime, with adaptive step size satisfying an error tolerance τ, the expected number of steps to integrate over $[0,T]$ obeys*

$$N \;\le\; C \cdot T \cdot M^{p/(p+1)} \cdot \tau^{-1/(p+1)}, \tag{5}$$

where $M = \sup_{t\in[0,T]} \|x^{(p+1)}(t)\|$ and C depends only on the method.

Sketch. The local truncation error of an order-p method is $\tau_n = \frac{h^{p+1}}{(p+1)!} x^{(p+1)}(\xi_n) + O(h^{p+2})$ for some $\xi_n \in [t_n, t_{n+1}]$. The optimal adaptive step satisfies $h^{p+1} M \approx \tau$, so $h \approx (\tau/M)^{1/(p+1)}$. The number of steps is $N \approx T/h$. See Hairer et al. [1993] §II.4. □

Connection to Jacobians. For an autonomous ODE, the first few time derivatives illustrate why Jacobian regularization can affect the local-error constant:

$$x''(t) = \frac{d}{dt} f(x(t)) = J(x(t)) f(x(t)), \tag{6}$$

$$x'''(t) = \frac{d}{dt}\big(J(x(t))f(x(t))\big) = J(x(t))^2 f(x(t)) + \big(DJ(x(t))[f(x(t))]\big) f(x(t)). \tag{7}$$

The second term in x''' contains Hessian information for f. Higher derivatives similarly involve products of Jacobians and higher derivatives of the vector field. Suppressing Jacobian norms can therefore reduce the local-error scale M, but $\|J\|_F$ alone does not determine M.

Practical tolerance caveat. Practical embedded adaptive solvers do not usually enforce a pure scalar tolerance. They control a weighted error norm whose components are scaled roughly as

$$\left| \frac{e}{a_{\text{tol}} + r_{\text{tol}} \, |x|} \right|. \tag{8}$$

Because our tolerance sweep co-scales a_{tol} and r_{tol} and re-trains a separate model in each tolerance cell, the observed exponent in §4.2 should be interpreted as consistency with expected solver behavior, not as a clean asymptotic estimate of $r_{\text{tol}}^{-1/(p+1)}$ for one fixed learned vector field.

A.4 Hutchinson estimator for the Frobenius norm squared

Theorem 4 (Hutchinson unbiasedness, after Hutchinson Hutchinson [1990])**.** *Let $A \in \mathbb{R}^{d\times d}$ and let $\epsilon \in \mathbb{R}^d$ be a random vector with $\mathbb{E}[\epsilon] = 0$ and $\mathbb{E}[\epsilon\epsilon^\top] = I_d$ (e.g., ϵ has i.i.d. Rademacher entries, or i.i.d. $\mathcal{N}(0,1)$). Then*

$$\mathbb{E}_\epsilon\Big[\|A^\top \epsilon\|^2\Big] \;=\; \|A\|_F^2. \tag{9}$$

Proof.

$$\mathbb{E}_\epsilon\Big[\|A^\top \epsilon\|^2\Big] = \mathbb{E}\left[\epsilon^\top A A^\top \epsilon\right] = \mathbb{E}\Big[\operatorname{tr}\Big(AA^\top \epsilon\epsilon^\top\Big)\Big] \tag{10}$$

$$= \operatorname{tr}\Big(AA^\top \, \mathbb{E}[\epsilon\epsilon^\top]\Big) = \operatorname{tr}(AA^\top) = \|A\|_F^2. \tag{11}$$

□

Variance bound. For Rademacher ϵ, $\operatorname{Var}[\|A^\top\epsilon\|^2] = 2\sum_{i\neq j}(AA^\top)_{ij}^2$, which is zero iff $AA^\top$ is diagonal. A K-sample average reduces variance by $1/K$.

Implementation. In automatic differentiation, $A^\top\epsilon$ is the vector–Jacobian product of f at the evaluation point with co-tangent vector ϵ, which costs a single backward pass through f regardless of d.

A.5 Van der Pol Jacobian calibration

The Van der Pol target used in the engagement analysis is

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = \mu(1 - x_1^2)x_2 - x_1. \tag{12}$$

Writing $f_{\text{vdp}}(x) = (f_1(x), f_2(x))^\top$, the partial derivatives are

$$\frac{\partial f_1}{\partial x_1} = 0, \qquad \frac{\partial f_1}{\partial x_2} = 1, \tag{13}$$

$$\frac{\partial f_2}{\partial x_1} = -2\mu x_1 x_2 - 1, \qquad \frac{\partial f_2}{\partial x_2} = \mu(1 - x_1^2). \tag{14}$$

Thus

$$J_{\text{vdp}}(x) = \begin{pmatrix} 0 & 1 \\ -2\mu x_1 x_2 - 1 & \mu(1 - x_1^2) \end{pmatrix}. \tag{15}$$

The squared Frobenius norm is the sum of squared entries:

$$\|J_{\text{vdp}}(x)\|_F^2 = 1 + (-2\mu x_1 x_2 - 1)^2 + \mu^2(1 - x_1^2)^2. \tag{16}$$

Table 5 in the main text evaluates (16) analytically on the same validation-time subsample used for learned-Jacobian diagnostics. This makes the calibration a true-target comparison on the diagnostic support rather than a separate global property of the Van der Pol vector field.

A.6 Ground-truth versus learned-trajectory penalty support

The implemented penalty samples states from the ground-truth trajectory distribution. Abstracting away the Hutchinson estimator, it is

$$R_{\text{gt}}(\theta) = \mathbb{E}_{(x,t)\sim\mu_{\text{gt}}}\left[\|J_{f_\theta}(x,t)\|_F^2\right]. \tag{17}$$

An augmented-state implementation instead integrates the penalty along the model's own trajectory:

$$R_{\text{model}}(\theta) = \mathbb{E}_{x_0}\int_0^T \|J_{f_\theta}(\hat{x}_\theta(t;x_0),t)\|_F^2 \, dt. \tag{18}$$

The two regularizers are close only when

$$\hat{x}_\theta(t;x_0) \approx x_{\text{gt}}(t;x_0) \tag{19}$$

on the time interval and initial-condition distribution of interest, and when the Jacobian norm does not vary too sharply between the two state supports. In the poor-fit $\mu = 50$ regime, the regularizer can therefore be weak for two distinct reasons: the learned vector field may have already collapsed to a low-Jacobian field, and the ground-truth penalty may be evaluated on states that the learned trajectory does not actually visit. The present experiment does not separate these mechanisms; it conflates low-Lipschitz collapse with penalty-support mismatch.

A.7 Penalty-engagement diagnostics

Write the regularized objective as

$$L(\theta) = L_{\text{task}}(\theta) + \lambda_J R(\theta). \tag{20}$$

A direct engagement diagnostic is the gradient-ratio

$$\eta_J(\theta) = \frac{\lambda_J \, \|\nabla_\theta R(\theta)\|}{\|\nabla_\theta L_{\text{task}}(\theta)\|}. \tag{21}$$

The regularizer can only shape training when $\eta_J(\theta)$ is non-negligible. Its direction also matters. A useful companion diagnostic is the gradient alignment

$$\cos(\nabla L_{\text{task}}, \nabla R) = \frac{\langle \nabla_\theta L_{\text{task}}(\theta), \nabla_\theta R(\theta) \rangle}{\|\nabla_\theta L_{\text{task}}(\theta)\| \, \|\nabla_\theta R(\theta)\|}. \tag{22}$$

If this cosine is near zero, the regularizer may be mostly orthogonal to the task update; if it is strongly positive in the convention of gradient descent on $L_{\text{task}} + \lambda_J R$, the two objectives pull in similar descent directions; if it is strongly negative, reducing the task loss and reducing the penalty are locally in conflict.

The back-of-envelope calculation in §4.3 compares loss scales, not gradient norms. It is a useful sanity check for why the $\mu = 50$ penalty may be hard to see, but it is not direct evidence about (21) or (22), especially under Adam's per-parameter rescaling. Logging η_J and the cosine during training would be the appropriate diagnostic for the proposed mechanism.

A.8 A-stability of implicit Euler

Theorem 5 (A-stability of implicit Euler, after Dahlquist Dahlquist [1963])**.** *The implicit Euler method $x_{n+1} = x_n + hf(x_{n+1})$ applied to the linear test equation $\dot{y} = \lambda y$ with $\mathrm{Re}(\lambda) < 0$ is stable for every $h > 0$.*

Proof. On $\dot{y} = \lambda y$, the scheme reduces to $y_{n+1} = y_n + h\lambda y_{n+1}$, so $y_{n+1} = (1 - h\lambda)^{-1} y_n$. For $\mathrm{Re}(\lambda) < 0$, $|1 - h\lambda|^2 = (1 - h\mathrm{Re}(\lambda))^2 + (h\mathrm{Im}(\lambda))^2 > 1$ for all $h > 0$, hence $|1 - h\lambda|^{-1} < 1$ and $|y_{n+1}| < |y_n|$. The scheme is therefore stable for all $h > 0$. Moreover $|1 - h\lambda|^{-1} \to 0$ as $h \to \infty$, so the method is also L-stable. □

Consequence. On a stiff system, the implicit Euler step size is governed only by accuracy (Mechanism A), not by stability (Mechanism B). Each step, however, requires solving a nonlinear equation; for a Newton-based implicit solver this is $O(\bar{K})$ Jacobian evaluations and linear solves per step.

A.9 The interaction prediction ⋆

We now combine the preceding results to derive the prediction that motivated the original H1 hypothesis. Let s index target-system stiffness and define the fractional NFE reduction at matched validation MSE as $\delta(s) := 1 - N_{\text{reg}}(s)/N_{\text{base}}(s)$. The original proposal used a single shrinkage factor β as a shorthand. That is too strong: Frobenius shrinkage, spectral-radius shrinkage, and

local-error shrinkage need not coincide. We therefore distinguish

$$\beta_F = \frac{\|J_{\text{reg}}\|_F}{\|J_{\text{base}}\|_F}, \qquad \beta_\rho = \frac{\rho(J_{\text{reg}})}{\rho(J_{\text{base}})}, \qquad \beta_M = \frac{M_{\text{reg}}}{M_{\text{base}}}, \tag{23}$$

where M is the local-error derivative scale from Theorem 3. In the experiments these quantities should be read as trajectory-local summaries on the validation diagnostic grid, not as global properties of the vector field.

By Theorem 3, on the smooth task

$$\frac{N_{\text{reg}}}{N_{\text{base}}} = \left(\frac{M_{\text{reg}}}{M_{\text{base}}}\right)^{p/(p+1)} = \beta_M^{p/(p+1)},$$

so the fractional NFE reduction is approximately $1 - \beta_M^{p/(p+1)}$ when the solve is accuracy-limited.

By Theorem 1, on the stiff task $N \propto T \cdot \rho(J)/C_p$, so

$$\frac{N_{\text{reg}}}{N_{\text{base}}} \approx \beta_\rho,$$

and the fractional NFE reduction is approximately $1 - \beta_\rho$ when the solve is stability-limited. Lemma 2 only gives $\rho(J) \le \|J\|_F$: reducing $\|J\|_F$ makes a reduction in $\rho(J)$ plausible, but it does not guarantee $\beta_\rho \approx \beta_F$.

Proposition 6 (Conditional interaction prediction)**.** *Suppose the smooth task is accuracy-limited, the stiff task is stability-limited, and regularization engages both the accuracy scale ($\beta_M < 1$) and the stability scale ($\beta_\rho < 1$) at matched validation MSE. Then the predicted ratio of fractional NFE reductions is*

$$\frac{\delta_{\textit{stiff}}}{\delta_{\textit{smooth}}} \approx \frac{1 - \beta_\rho}{1 - \beta_M^{p/(p+1)}}. \tag{24}$$

A positive interaction requires this ratio to exceed one; equivalently, the spectral-radius engagement in the stability-limited regime must be strong enough relative to the accuracy-scale engagement in the accuracy-limited regime. It is not implied by $\beta_F < 1$ alone.

Empirical caveat. The antecedent of Proposition 6 is exactly what fails in this experiment. At $\mu = 50$, the Frobenius penalty is operationally inert because the learned field is already low-Jacobian; at $\mu = 10$, where Frobenius engagement is visible, the learned dynamics do not show a stability-limited step-size restriction. The proposition therefore does not prove the empirical trade-off. It states a conditional mechanism for a hypothetical regime in which fit, penalty engagement, and learned stiffness co-occur.

A.10 Cost decomposition for implicit Runge–Kutta

Proposition 7 (Implicit cost decomposition)**.** *For an s-stage implicit Runge–Kutta method run for N outer steps, with average Newton iteration count $\bar{K}$ per step, the cumulative work is*

$$W \;=\; N \cdot \bar{K} \cdot \big[s \cdot W_f \;+\; W_J \;+\; W_{lin}\big], \tag{25}$$

where W_f is the cost of one f evaluation, W_J the cost of one Jacobian evaluation, and W_{lin} the cost of a single linear solve of dimension $d \cdot s$.

Equation (25) is the cost model for serious Newton-based stiff solvers such as implicit Runge–Kutta, BDF, or Radau-type methods. In that setting, the NFE count alone (which is $N \cdot \bar{K} \cdot s$ for implicit Runge–Kutta) is not sufficient to characterize wall-clock cost; the Jacobian and linear-solve terms can dominate on small d where dense linear algebra is cheap but W_J is non-trivial.

This is not the exact cost model for the empirical H3 baseline: `torchdiffeq.implicit_adams` uses fixed-point iteration rather than Newton solves. Its leading per-step cost is closer to $N \cdot \bar{K}_{\text{fp}} \cdot W_f$, with no explicit Jacobian or linear-solve term exposed to the user. It also does not represent a full comparison against modern A- or L-stable Newton-based stiff solvers. Proposition 7 therefore motivates why implicit methods can have hidden costs, but it does not describe the exact implementation used in the H3 comparison.

B Reproducibility Checklist

This checklist records settings not already stated in §3; we do not duplicate model, optimizer, training, or solver descriptions that appear in the main text.

1. **Software.** Python 3.10.5, PyTorch 2.11.0, torchdiffeq 0.2.5 [Chen, 2018], NumPy 2.2.6, SciPy 1.15.3, matplotlib 3.10.9. Hardware: Apple Silicon laptop, CPU-only training. Total wall time for the 150-run sweep inventory: ≈ 9 hours.
2. **Wall-clock measurement protocol.** Each cell's reported wall-clock per forward pass is a single timed call to `model(x0_val, t)` under `torch.no_grad()`, with no warm-up pass. On CPU this is noisy at the millisecond level; we recommend reproducing with at least three timed passes per cell if precise wall-clock numbers matter to the downstream conclusion.
3. **IC generator ordering.** For each seed, a single `torch.Generator(seed)` is used to draw the 256 training x_0 samples and then the 64 validation samples (disjoint, consecutive draws from the same generator). The global PyTorch RNG is independently seeded with the same integer for model initialisation and Hutchinson noise.
4. **Penalty sampling distribution.** 32 (t, x) pairs per training step are drawn uniformly from the *flat* $(T \cdot B)$ index of ground-truth training trajectories, so time and initial-condition indices are sampled jointly with equal weight. This implicitly over-samples slow-manifold portions of the Van der Pol trajectory where the uniform-in-t grid is dense.

5. **Tolerance sweep coupling.** r_{tol} and a_{tol} are both varied in the tolerance sweep: $(r_{\text{tol}}, a_{\text{tol}}) \in \{(10^{-3}, 10^{-5}), (10^{-5}, 10^{-7}), (10^{-7}, 10^{-9})\}$. The fitted exponent α in §4.2 therefore reflects sensitivity to both tolerances jointly. The tolerance override is applied before training, so these cells are train-and-evaluate tolerance settings rather than an eval-only sweep on one fixed trained model.

6. **Solver-registry mutation.** Tolerance overrides are implemented by in-process mutation of `SOLVER_REGISTRY` in `scripts/train_single.py`. Each cell runs in a fresh subprocess (via `run_matrix.py`) so the mutation does not leak across cells.

7. **Statistical tests.** Paired Wilcoxon signed-rank with the *wilcox* zero-handling convention; rank-biserial correlation as effect size. Bonferroni correction is applied at family-wise $\alpha = 0.05$ across the H3 set ($m = 3$). With each H3 comparison hitting one-sided $p = 0.031$ uncorrected, none survives Bonferroni at $\alpha/m = 0.017$ when treated as a family. We therefore report the consistent direction across μ as the substantive finding, not the per-comparison significance after correction.

8. **Result provenance.** Result filenames encode task, solver, λ_J, and seed; full metadata such as μ, K, tolerance overrides, and the output directory for follow-up sweeps must be read from the JSON `config` block and directory structure. A cleaner archival release would include a manifest or filenames containing every experimental factor.

9. **Total runs.** 150 trained runs across all sweeps, summarised in Table 1 of the main text.